

Does the Documentation of Design Pattern Instances Impact on Source Code Comprehension? Results from Two Controlled Experiments

Carmine Gravino¹, Michele Risi¹, Giuseppe Scanniello², Genoveffa Tortora¹

¹Facoltà di Scienze MM. FF. NN.

University of Salerno

Via Ponte Don Melillo, I-84084, Fisciano (SA), Italy

{gravino, mrisi, tortora}@unisa.it

²Dipartimento di Matematica e Informatica

University of Basilicata

Viale dell'Ateneo, I-85100, Potenza, Italy

e-mail: giuseppe.scanniello@unibas.it

Abstract—We present the results of a controlled experiment and a differentiated replication that have been carried out to assess the effect of the documentation of design patterns on the comprehension of source code. The two experiments involved Master Students in Computer Science at the University of Basilicata and at University of Salerno, respectively. The participants to the original experiment performed a comprehension task with and without graphically-documented design patterns. Textually documented design patterns were provided or not to the participants to perform a comprehension task within the replication. The data analysis revealed that participants employing graphically documented design patterns achieved significantly better performances than the participants provided with source code alone. Conversely, the effect of textually documented design patterns was not statistically significant. A further analysis revealed that the documentation type (textual and graphical) does not significantly affect the performance, when participants correctly recognize design pattern instances.

Keywords- *Design Patterns, Controlled Experiment, Maintenance*

I. INTRODUCTION

Maintenance is an important phase for software systems developed by using any software life cycle model and programming language. Software maintenance is needed to ensure that a software system continues to satisfy users' requirements [3]. Whatever is the required maintenance operation (e.g., corrective, perfective, and adaptive [19]), maintainers must spend considerable time to read and comprehend source code written by others. The availability of design choices should provide a better support to accomplish maintenance operations, thus reducing the needed effort and positively affecting the efficiency with which maintainers perform these operations [8].

In the object-oriented development field, design patterns are recognized as a means that “can improve the documentation and maintenance of existing systems by furnishing an explicit specification of class and object interactions and their underlying intent” [8]. However, a few

empirical investigations have been conducted to verify the support provided by the documentation of design pattern instances in the comprehension of source code (e.g., [10][13]).

In this paper, we present a controlled experiment and a differentiated replication to assess the effect of using two different types of representation for documenting design pattern instances¹ on the comprehension of source code. The participants involved in the original experiment were 24 students of the Master program in Computer Science at the University of Basilicata. The participants were divided in two groups each composed of 12 students and were asked to accomplish an experimental task on the source code of a chunk of JHOTDRAW v5.1 [11]. Each design pattern instance was graphically documented using a UML class diagram. We gave the documented instances and the source code to the participants within the first group (treatment group). The participants of the second group accomplished the comprehension task with the source code alone (control group). The data analysis revealed that: graphically documented design pattern instances significantly reduce the time and significantly improves the efficiency of the subjects to perform the comprehension tasks.

We conducted the replication in a different context with students from the University of Salerno enrolled to the Master course in Computer Science. These students used textually-documented design pattern instances on the same source code as the original experiment. We performed a data analysis to assess the contribution provided by this different kind of documentation and with respect to [18] we give the following contributions:

- analyzing/studying the effect of a new performance indicator for the participants, i.e., the comprehension;
- investigating the impact of graphical and textual design pattern instances;

¹ A design pattern includes a name, an intent, a problem, its solution, some example, and so on [8]. In the paper, we focus on the solutions (also known as design motifs [9]) and we will refer to them as design pattern instances.

- verifying how the capability in correctly identifying design pattern instances affects participants' performances;
- investigating the relationship between the participants' performances and the number and the kind of design pattern instances;
- analyzing the source of information that participants used to perform comprehension tasks both using and not using design pattern instances.

The paper is structured as follows: Section II presents related work, while Section III provides details on the planning of the controlled experiments. The results are analyzed and discussed in Section IV, while threats to validity are presented in Section V. Final remarks and future direction conclude the paper.

II. RELATED WORK

This section discusses the related literature concerning controlled experiments and case studies aimed at verifying the support of design patterns for executing comprehension tasks and for maintaining software. Prechelt *et al.* [15] investigated whether design pattern instances explicitly and textually documented in the source code (through comment) could improve the performance of maintainers in performing comprehension tasks with respect to a well-commented program without explicit reference to design patterns. The investigation involved 74 German graduate students and 22 USA undergraduate students, who performed maintenance operations on Java and C++ code, respectively. The performed analysis revealed that maintenance tasks supported by explicitly-documented design patterns were completed faster or with fewer errors. The most remarkable difference with our work is that we also analyze the effect of design pattern instances documented in a graphical way.

Jeanmart *et al.* [10] performed an experiment to collect data on the impact of the Visitor pattern on comprehension and modification tasks supported by class diagrams. Thus, differently from our study, they did not consider tasks on source code. Three programs and 24 developers were involved in their study and an eye-tracker was used to collect data. The performed analysis suggested that no significant difference was found between class diagrams with, without, or with a modified representation of the Visitor when performing comprehension tasks. As for modification tasks, the results revealed that developers performed in less time on diagrams where the Visitor pattern was represented with the canonical representation suggested in by Gamma *et al.* [8].

Differently, Porras and Guéhéneuc [14] analyzed four different graphical representations of design patterns, by focusing on comprehension tasks with diagrams. In particular, they performed an empirical study to compare three representations of design patterns with a canonical representation in terms of UML class diagrams. The participants performed comprehension tasks, such as identifying composition, role, and participation within the design patterns. Similar to [10], the authors exploited eye-trackers to collect data on the developers' effort during the execution of the experiment. The analysis of the collected data revealed that the UML class diagrams enhanced with

stereotypes better supported the participants in the identification of composition and role of the UML notation without stereotypes. Moreover, the UML notation and UML class diagrams enhanced with stereotypes allowed participants to obtain better results, when participants were asked to identify the classes participating in a design pattern.

Researchers have also analyzed the impact of design patterns on the evolution of software systems, focusing their investigations on the roles that the classes play in design pattern instances. For example, Bieman *et al.* [4] and Di Penta *et al.* [7] showed that some roles were more change prone than others. Differently, Vokac [20] analyzed and compared the defect rates of classes participating in design pattern instances with respect to those not involved. These studies showed that classes involved in design pattern instances were less defect prone than others.

Finally, Khomh and Guéhéneuc [13] carried out an empirical study to analyze the impact of the design patterns defined by Gamma *et al.* [8] on ten software quality characteristics. The results showed that instances of these patterns did not always improve the quality of the software (e. g., expandability and understandability).

III. EXPERIMENT DESIGN

In this section, we present the design of the experiment and its replication. We adopted the method proposed by Wohlin *et al.* [21]. For readability reasons, we first present the independent and dependent variables and then the tested null hypotheses. We highlight and discuss the differences between the original experiment and the replication². For replication purposes, we make available on the Web (at www.scienzemfn.unisa.it/scanniello/DP_1/) an experimental package and the raw data.

A. Context

We conducted the experiment within a laboratory at the University of Basilicata (Italy) with 24 students of the Master program in Computer Science. The experiment represented an optional activity of an Advanced Software Engineering course. We conducted the replication within a laboratory at the University of Salerno (Italy) with 17 students of the Master Program in Computer Science. The experiment represented an optional activity of an advanced Database System Modeling course.

All the involved participants had basic software engineering knowledge. In particular, they knew the basics of requirements engineering, high- and low-level design of object-oriented software systems based on UML, software development, and software maintenance. They had, however, a limited experience in developing and maintaining nontrivial software systems.

The participants to the original experiment (UniBas in the following) and its replication (UniSa in the following) accomplished a comprehension task on the same source

² It was a differentiated replication because variations in essential aspects of the experimental conditions have been introduced [1]. This kind of replications can be conducted to identify potentially important experimental factors that affect the results.

Q6. What are the concrete class/es, abstract class/es, and interface/s involved in the implementation of the functionality: Adding a figure to another figure?				
How much do you trust your answer ⁺ ?				
☐ Unsure	☐ Not sure enough	☐ Sure Enough	☐ Sure	☐ Very Sure
How do you assess the question ⁺ ?				
☐ Very difficult	☐ Difficult	☐ On average	☐ Simple	☐ Very Simple
What is the source of information used to answer the question ⁺ ?				
☐ Design Pattern instances (DPI)	☐ Previous Knowledge (PK)	☐ Comment (Com)	☐ Source Code (SC)	
+ Mark only one answer				

Figure 1 A question example from the comprehension questionnaire.

code, which was selected from a well-known open source Java software system, JHOTDRAW v5.1 [11]. In particular, we selected a vertical slice of this system that included: (i) a nontrivial number of design pattern instances and (ii) well-known and widely adopted design patterns. In the selection process, we have also taken into account a tradeoff between the complexity of the implemented functionality and the effort to comprehend it (less than 3 hours when participants used source code alone).

The comment of the source code was translated from English into Italian. We translated the comments in order to avoid biasing the results because different participants may have different familiarity with English. Further, we removed any reference to the design pattern instances from the comment and the identifiers when participants performed the comprehension task with the source code alone (e.g., the name of the class *CompositeFigure* in Figure 2 was turned into *ArrayFigure*). Therefore, participants worked on a modified version of JHOTDRAW.

Table I shows some descriptive statistics of the source code used in the experiments. One of the authors manually detected the design pattern instances in source code. To this end, he used the documentation of JHOTDRAW and the PMART³ data set. It is worth mentioning that some instances may share the same classes or interfaces.

TABLE I DESCRIPTIVE STATISTICS OF EXPERIMENTAL TASK

# Line Of Code	1326
# Classes	26
# Line of comment	823
# State Pattern	2
# Adapter Pattern	1
# Strategy Pattern	1
# Decorator Pattern	1
# Composite Pattern	1
# Observer Pattern	1
# Command Pattern	1
# Template Method Pattern	1
# Prototype Pattern	1

B. Selected Variables and Hypotheses Formulation

To analyze the participants' performances, we selected three dependent variables: *Effort*, *Comprehension*, and *Efficiency*. *Effort* is measured as the time (expressed in

minutes) to accomplish the comprehension task. Similar to [16], this time was recorded directly by each participant noting down his/her start and stop time.

With regard to the Comprehension dependent variable, we used a comprehension questionnaire that participants filled out to assess their comprehension of the source code. The questionnaire was composed of 14 open questions. We divided these questions in three groups to let participants take a break if needed when passing from a group of questions to the other one. This choice was taken for reducing the fatigue effect on the participants' performances. The supervisors controlled each taken break to avoid the exchange of information among the participants and then bias the results.

We defined the questions of the comprehension questionnaire to assess several aspects related to the comprehension of the source code. For each question, the participants had to specify a list of answers (e.g., class and interfaces names). Figure 1 shows a sample question of the comprehension questionnaire used in both the experiments.

To quantitatively evaluate the answers provided by each participant, we adopted an information retrieval based approach. Similar to [16][17], we used the precision and recall measures to assess, respectively, the correctness and completeness of the answers for each question:

$$precision_{s,i} = \frac{|answers_{s,i} \cap correct_i|}{|answers_{s,i}|} \quad recall_{s,i} = \frac{|answers_{s,i} \cap correct_i|}{|correct_i|}$$

where $answers_{s,i}$ is the set of string items provided as answer to the question i by the participant s , while $correct_i$ was the known correct set of items expected for the question i . We have identified the set of correct answers for all the questions before conducting the experiments. We have obtained a balance between correctness and completeness using the harmonic mean of precision and recall for each question. Finally, to quantitatively assess the comprehension a subject achieved on the source code, we have computed the overall average of the F-measures of all the questions of the comprehension questionnaire:

$$F - measure_s = \frac{2 \times precision_s \times recall_s}{precision_s + recall_s}$$

The Comprehension dependent variable is measured as F-measure %, while Efficiency is computed by dividing Comprehension by Effort. One of the authors calculated the Comprehension and Efficiency values for each participant.

³ <http://www.ptidej.net/downloads/pmart/>

He was involved in the definition of neither the experimental object nor the comprehension questionnaire.

Regarding the comprehension questionnaire, we also collected data on the source of information used by the participants to answer each question. In particular, we asked the participants who accomplished the task with source code complemented with design pattern instances to specify for each question whether the answer was derived using: *(DPI) Design Pattern Instances* (graphically documented for UniBas and textually documented for UniSa), *(PK) Previous Knowledge*, *(Com) source code Comment*, or *(SC) Source Code*. The participants who accomplished the task using the source code alone chose among: *(PK) previous knowledge*, *(Com) source code comment*, and *(SC) source code*. The confidence level (e.g., “I’m sure”) and the degree of complexity (e.g., “difficult”) were also sought for each question answered (see Figure 1).

In our experimentation, the *control group* is the group using the source code with no documented design pattern instances, while the *treatment group* is the source code with the documentation (graphically or textually) of the design pattern instances. Thus, the only independent variable is *Method* (i.e., the main factor), which is a nominal variable with two possible values: DP (documented Design Pattern instances) and NO_DP (NO documented Design Pattern instances).

To analyze the data, we also considered the following cofactors: DT (Documentation Type) and DPIA (Design Pattern Identification Ability). DT indicates the kind of documentation used to represent design pattern instances. It is a nominal variable with two possible values: GD (Graphically Documented) and TD (Textually Documented). DPIA indicates the participants that correctly or incorrectly identified design pattern instances to answer a given question of the comprehension questionnaire. This cofactor is a nominal variable with two possible values: DPCI (i.e., Design Patterns Correctly Identified) and DPnCI (i.e., Design Patterns not Correctly Identified).

We have then investigated the following null hypotheses:

- **Hn0_X.** The participants who use the documentation of design pattern instances (GD or TD) do not achieve significantly better results in terms of X (where X is a measure: Effort, Comprehension, or Efficiency).
- **Hn1_X.** There is not a significant difference with respect to the X values when participants use GD or TD.
- **Hn2_X.** There is not a significant difference with respect to the X values when DPCI participants use TD or GD.
- **Hn3_X.** The DPCI participants do not achieve significantly better X values than the participants who use NO_DP.
- **Hn4_X.** The DPCI participants do not achieve significantly better X values than the DPnCI participants.

Hn0_X, Hn3_X, and Hn4_X are one-tailed, while the others are two-tailed. The goal of the statistical analysis is to reject the defined null hypotheses and accepting the alternative ones (i.e., Ha0_X, Ha1_X, Ha2_X, Ha3_X, and

Ha4_X), which can be easily derived (e.g., Ha0_X: The participants who use the documentation of design pattern instances significantly achieve better results in terms of X).

C. Experiment Design

We used the *one factor with two treatments design* [21]. The design set up of both the experiment uses the same *experimental object* (i.e., the selected chunk of JHOTDRAW) for the two methods, namely DP and NO_DP. We used the results of a pre-questionnaire (see Section III.D) to equally distribute high and low ability participants in two groups: A and B. The participants within the group A used DP, while the ones in the group B used NO_DP. Thus, each participant used either DP or NO_DP to accomplish the task. With respect to the original experiment, the participants within each group were 12, while in the replication 9 participants were assigned to A and 8 to B.

This design was chosen because we were interested in using a realistic task (as much as possible) in terms of both size and complexity, so mitigating external validity threats. This design choice did not enable the use of more sophisticated experiment design for studying the effect of other factors, such as the counterbalanced design. The use of a different experiment design with non-trivial experimental objects may bias the results introducing a factor difficult to be controlled, i.e., mental fatigue.

Beside the participants involved in the two experiments, the other difference regarded how we documented the design pattern instances. In the original experiment, each instance was represented in terms of a UML class diagram. For example, Figure 2(a) shows an instance of the Composite design pattern [8] within the source code of the task. As far as the replication is concerned, design pattern instances were documented as shown in Figure 2(b), where the same instance of the design pattern reported in Figure 2(a) is textually represented as a comment in the source code. We added the documentation of the design pattern instances to the comment of JHOTDRAW translated into Italian. The shown instance (both textual and graphical) could be used to answer the question of Figure 1. In particular, the expected correct answers were: (i) *ArrayFigure*, (ii) *GenericFigure*, (iii) *Figure*, and (iv) *FigureChangeListener*. The identification of the instance of the Composite design pattern should better support the participants in answering the question, because the Composite pattern is used to create hierarchical and recursive tree structures of objects.

D. Execution

We performed a pilot experiment some days before the original experiment with a research fellow and a Ph.D. student at the University of Salerno. The pilot results indicated that 3 hours were, on average, sufficient to accomplish the task using or not using design pattern instances. The participants involved in the pilot also indicated some minor issues in the experimental material that we addressed before conducting the experiments.

Regarding the experiments, all the participants attended an introductory lesson in which we presented detailed instructions on the task to perform. We only highlighted the

goal of the experiments, while details on the experimental hypotheses were not provided. We asked the participants to fill out a pre-questionnaire to gather information about passed exams, industrial working experience, and grade point average. Before the experiment the participants did not know JHOTDRAW.

To perform the comprehension task, we provided each participant with a computer. We asked the participants to use the following experimental procedure for each group of questions: (i) specifying name and start-time; (ii) answering the questions browsing the source code (using a never used text editor); and (iii) marking the end-time. We did not suggest any approach to accomplish the task.

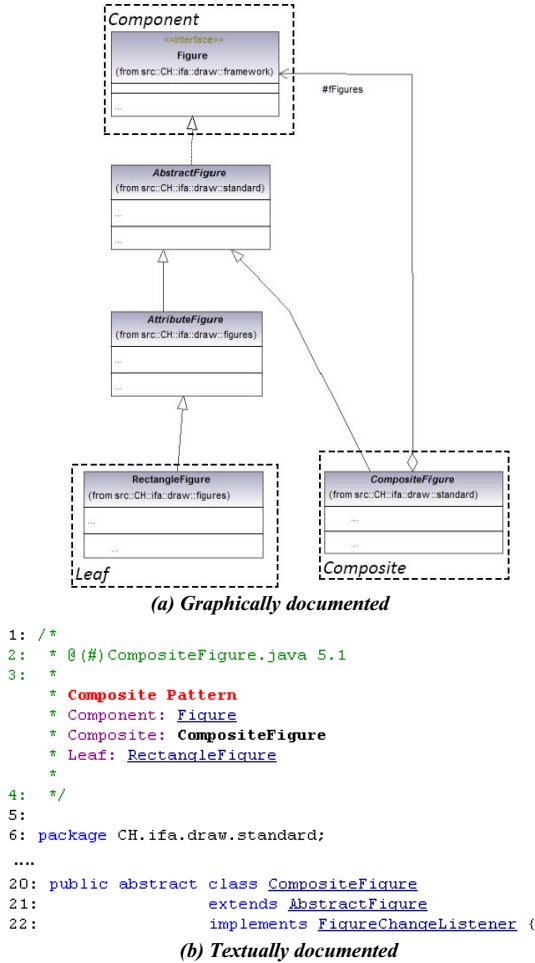


Figure 2 A sample of design pattern instance

We provided the participants with a paper copy of the following experimental material: (i) the comprehension questionnaire and (ii) a post-experiment survey questionnaire (see Table II). With regard to the post-experiment survey questionnaire, the participants that used DP had to answer all the questions in Table II, while the other participants answered the questions from S1 to S5. The goal of this questionnaire was to gain insights (e.g., on the experimental object) to better explain the results.

TABLE II POST-EXPERIMENT SURVEY QUESTIONNAIRE

Id	Question	Possible Answers
S1	I had enough time to perform the task	(1-5)
S2	The tasks I performed were perfectly clear to me	(1-5)
S3	The task objectives were perfectly clear to me	(1-5)
S4	The comment included in source code were clear	(1-5)
S5	I found useful the organization in three parts of the questions	(1-5)
S6	The design pattern instances were useful to answer the questions	(1-5)
S7	The design pattern included in the system were well documented	(1-5)
S8	How much time (in terms of a percentage) did you spend to analyze the source code?	(A-E)
S9	How much time (in terms of a percentage) did you spend to analyze the design pattern instances?	(A-E)
1 = Strongly agree, 2 = Agree, 3 = Neutral, 4 = Disagree, 5 = Strongly disagree		
A. <20%; B. >=20% and <40%; C. >=40% and <60%; D. >=60% and <80%; E. >=80%		

The participants who used DP in UniBas (i.e., GD) were also provided with the paper copy of a document where the design pattern instances were graphically reported (see for example Figure 2(a)). The participants that used DP in UniSa (i.e., TD) were provided with source code that included the references to the design pattern instances in the comment of the source code (see for example Figure 2(b)).

E. Data Analysis

To investigate the null hypotheses, we adopted non-parametric tests due to the sample size and mostly the non-normality of the data. In particular, we used the Mann-Whitney test [6] due to the design of the experiments (only unpaired analyses are possible) and its robustness [21].

This test allows the presence of a significant difference between dependent or independent groups to be verified, but it does not provide any information about this difference [12]. We therefore used the Cohen's d [5] effect size to obtain the standardized difference between two groups that can be considered negligible for $|d| < 0.2$, small for $0.2 \leq |d| < 0.5$, medium for $0.5 \leq |d| < 0.8$, and large for $|d| \geq 0.8$. In the context of unpaired analyses, Cohen's d effect size can be calculated as the difference between the means, divided by the pooled standard deviation of both groups.

In all the performed statistical tests, we decided (as custom) to accept a probability of 5% of committing Type-I-error [21], i.e., of rejecting a null hypothesis when it is actually true.

IV. ANALYSIS AND DISCUSSION

Table III and Table IV show respectively some descriptive statistics (i.e., median, mean, and standard deviation) of UniBas and UniSa, grouped by Method, Effort, Comprehension, and Efficiency. These statistics show that the UniSa participants spent less time to accomplish the task, while the Comprehension achieved by the participants within UniBas and UniSa using either DP or NO_DP is nearly the same. Thus, the UniSa participants had on average a higher Efficiency. We can also observe that the participants to

UniBas spent on average less time when using DP rather than NO_DP. Further, they achieved on average higher values of Comprehension and Efficiency when using DP with respect to NO_DP. For UniSa, the results achieved with DP and NO_DP can be considered comparable, even if the participants using DP obtained on average better Comprehension and Efficiency.

Figure 3 shows the boxplots for Effort, Comprehension, and Efficiency of both the experiments. Regarding UniBas, the differences in favor of DP for Effort and Efficiency are more evident (see also the descriptive statistics). Differently, a slight difference in favor of DP is shown for Comprehension.

The boxplots of UniSa suggest that the results in terms of Effort are similar, while the Comprehension values achieved with DP are better distributed and have a higher median than those of NO_DP. Differently, we can observe that the median of DP is higher on Efficiency and the box length and tails are more skewed for NO_DP.

Table V and Table VI show descriptive statistics (i.e., median, mean, and standard deviation) of UniBas and UniSa, grouping the observations by DPCI and DPnCI and the dependent variables. The results indicate that the participants achieved on average better values for the dependent variables

TABLE III DESCRIPTIVE STATISTICS FOR DP AND NO_DP (UniBas)

Dependent Variable	DP			NO_DP		
	Med.	Mean	Std. Dev.	Med.	Mean	Std. Dev.
Effort	157	158	26	182	188	17
Comprehension	49	49	13	46	46	9
Efficiency	0.32	0.31	0.07	0.24	0.25	0.04

TABLE IV DESCRIPTIVE STATISTICS FOR DP AND NO_DP (UniSa)

Dependent Variable	DP			NO_DP		
	Med.	Mean	Std. Dev.	Med.	Mean	Std. Dev.
Effort	97	102	26	101	98	14
Comprehension	49	47	10	42	45	8
Efficiency	0.53	0.48	0.13	0.45	0.46	0.08

TABLE V DESCRIPTIVE STATISTICS FOR DPCI AND DPnCI (UniBas)

Dependent Variable	DPCI			DPnCI		
	Med.	Mean	Std. Dev.	Med.	Mean	Std. Dev.
Effort	11	12	4	11	11	5
Comprehension	71	67	29	46	42	31
Efficiency	5.80	6.83	4.97	4.44	4.49	3.77

TABLE VI DESCRIPTIVE STATISTICS FOR DPCI AND DPnCI (UniSa)

Dependent Variable	DPCI			DPnCI		
	Med.	Mean	Std. Dev.	Med.	Mean	Std. Dev.
Effort	6	6	3	7	8	3
Comprehension	80	64	36	50	45	36
Efficiency	12	16	16	7	7	6

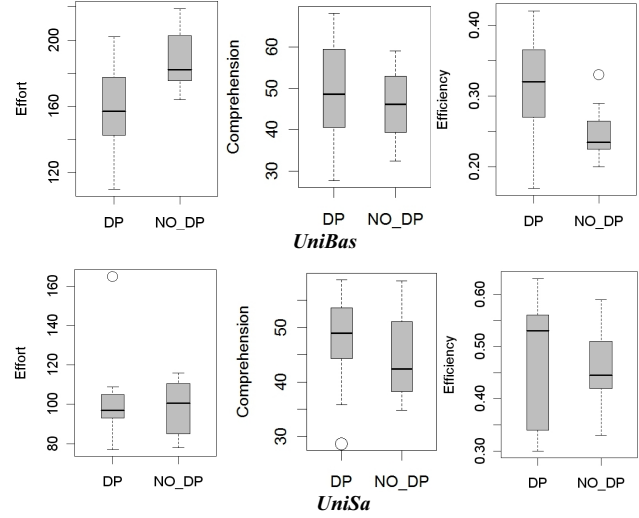


Figure 3 Boxplots of Effort, Comprehension, and Efficiency.

(with the exception of UniBas on Effort) both using TD and GD when they correctly detected the instances of the design patterns. This finding is interesting from both the researcher and the project manager points of view: *is it important to provide means to effectively support maintainers in the identification of design pattern instances?* We preliminarily investigated this point and the achieved results are presented in the following.

A. Hypotheses Testing

The results of the Mann-Whitney test performed to assess the null hypotheses related to Hn0_X are summarized in Table VII, together with the Cohen *d* effect size. The results show that Hn0_Effort and Hn0_Efficiency can be rejected for UniBas because the p-values are less than 0.01, with a large effect size. Differently, Hn0_Comprehension cannot be rejected (p-value = 0.30). **This result together with the descriptive statistics suggests that participants achieved a better comprehension with DP but the effect of the documentation of the design pattern instances is not statistically significant.**

Regarding UniSa, we did not reject the null hypotheses related to Hn0_X as the p-values of Table VII show. Further, we obtained on all the dependent variables a small or a negligible effect size. Thus, **the participants achieved better performances using DP (see Table IV) but the effect of the documentation is not statistically significant.**

Table VIII shows the results of the null hypotheses related to Hn1_X. **The Mann-Whitney test results revealed that the participants who used TD spent significantly less time to accomplish the comprehension task** with a large effect size. Further, these participants achieved significant better Efficiency values (the practical difference is small), thus indicating that the efficiency increases when design pattern instances are textually documented. For Comprehension the effect of the instance documentation is not statistically significant. The participants that used GD achieved on average a better comprehension level of the source code (see Table I and Table II).

TABLE VII STATISTICAL TEST RESULTS FOR DP VS NO_DP (Hn0)

Exp.	Hypothesis	p-value	Effect Size
UniBas	Effort	<0.01	d=-1.36
	Comprehension	0.30	d=0.24
	Efficiency	<0.01	d=1.12
UniSa	Effort	0.42	d=19
	Comprehension	0.27	d=29
	Efficiency	0.27	d=20

TABLE VIII STATISTICAL TEST RESULTS FOR GD VS TD (Hn1)

Hypothesis	p-value	Effect Size
Effort	<0.01	d=1.01
Comprehension	0.89	d=0.03
Efficiency	<0.01	d=-0.47

TABLE IX STATISTICAL TEST RESULTS FOR GD VS TD (Hn2)

Hypothesis	p-value	Effect Size
Effort	0.01	d=1.62
Comprehension	0.96	d=0.06
Efficiency	0.01	d=-0.76

TABLE X STATISTICAL TEST RESULTS FOR DPCI VS NO_DP (Hn3)

Exp.	Hypothesis	p-value	Effect Size
UniBas	Effort	<0.01	d=-0.42
	Comprehension	<0.01	d=0.60
	Efficiency	<0.01	d=0.68
UniSa	Effort	0.05	d=-0.36
	Comprehension	0.02	d=0.49
	Efficiency	<0.01	d=0.61

TABLE XI STATISTICAL TEST RESULTS FOR DPCI VS DPnCI (Hn4)

Exp.	Hypothesis	p-value	Effect Size
UniBas	Effort	0.84	d=-0.42
	Comprehension	<0.01	d=0.80
	Efficiency	<0.01	d=0.53
UniSa	Effort	0.03	d=-0.48
	Comprehension	0.01	d=0.54
	Efficiency	<0.01	d=0.72

Table IX shows the results of the statistical analysis of the null hypotheses Hn2_Effort, Hn2_Comprehension, and Hn2_Efficiency. The results of the Mann-Whitney test indicate a positive effect of TD with respect to Effort and Efficiency, when participants correctly identified design pattern instances to answer the questions of the comprehension questionnaire. The practical difference is large on Effort and medium on Efficiency. The statistical investigation did not enable to reject Hn2_Comprehension. However, the descriptive statistics (see Table V and Table VI) indicate that the participants who used GD achieved on average a better comprehension of the source code.

Table X shows the results of the Mann-Whitney test on Hn3_X. Regarding UniBas, the null hypotheses Hn3_Effort, Hn3_Comprehension, and Hn3_Efficiency can be rejected (all the p-values are less than 0.01) with a small effect size on Effort and a medium effect size on both Comprehension and Efficiency. Thus, the participants who correctly identified design pattern instances and used DP achieved significantly better Comprehension and Efficiency values than the participants who used NO_DP. The participants significantly spent less time when using DP. We achieved similar results on UniSa with respect to Comprehension and Efficiency. The null hypothesis Hn3_Effort was not rejected.

TABLE XII RESULTS OF THE ANALYSIS BY QUESTION

Exp.	Variable	Questions	Design Patterns
UniBas	Effort	Q1, Q2	Prototype, Composite, Observer, State/Strategy, Template Method
	Comprehension	Q2, Q3, Q9	Prototype, Composite, Observer, State/Strategy, Template Method, Command
	Efficiency	Q2, Q3, Q5	Prototype, Composite, Observer, State/Strategy, Template Method
UniSa	Effort	Q9	Command
	Comprehension	Q9	Command
	Efficiency	Q9	Command

Table XI summarizes the results of the Mann-Whitney test on Hn4_X. As almost expected, the participants within DPCI that used DP achieved significantly better performances than the ones of DPnCI in both the experiments (the practical differences range from small to large). This is always true except for UniBas on Effort (p-value = 0.84). This indicates that the participants needed significantly more time in case of GD to identify correctly the instances.

B. Further Analyses

We present two further analyses to: (i) explore the support provided by the kinds of design patterns and (ii) analyze whether the participants perceived design pattern instances as a relevant or predominant source of information to answer the comprehension questionnaire.

1) Analysis by Question

To understand the impact of the use of a specific kind of design pattern on the source code comprehension, we have also compared the participants' performance of DPCI and DPnCI on each question of the comprehension questionnaire. This further investigation was possible due to the experiment design and to the fact that the participants needed to identify one or more design pattern instances to answer each question (see for example Figure 1). Table XII also shows the questions for which the DPCI participants achieved significant better results than DPnCI participants. We can observe that the DPCI participants to UniBas achieved significant better results than DPnCI ones on two or three questions. Differently, the DPCI participants to UniSa achieved significantly better results on all the dependent variables with respect to Q9.

The results of UniBas (see fourth column of Table XII) also indicated that the design patterns that better supported the comprehension were: Prototype, Composite, Observer, State/Strategy, Template Method, and Command. However, some of these design patterns (i.e., Composite, Template Method) were also involved in other questions (i.e., Q6, Q10, Q12, Q13, and Q14) for which the DPCI participants did not obtain significant better results. Differently, the results of UniSa indicated that the Command design pattern better supported the participants in the source code comprehension. In fact, to answer Q9, only the correct

identification of the Command instances was needed. Further details are not provided for space reason.

A possible justification for these results could be related to the kind of documentation for the design pattern instances within UniBas and UniSa. This point can be considered interesting from the researcher's point of view because it could be useful to study how the presence, the interaction, and the kind of documentation of design pattern instances affect the performance of software engineers in the execution of comprehension tasks. This is a possible future direction for our research.

2) Source of Information

Similar to [17], we also analyzed the source of information (see Section III.B) the participants declared to use for performing the comprehension task. We used the mosaic plots of Figure 4. These plots graphically represent the frequency of the sources of information used by the participants to answer the questions. We highlighted in green the frequency of the participants that selected DPI as source of information. We can observe that for both UniBas and UniSa design patterns were considered the second source of information. The first source of information was SC.

In our analysis, we have also considered two levels of importance for the source of information: relevant (design pattern instances are more important than the average source of information) and predominant (design patterns are the first most important source of information). Therefore, we have investigated whether: (i) the proportion of questions where instances represented the source of information used to answer is equal or lower than the average proportion of all information sources; and (ii) the proportion of questions where instances represented the source of information used to answer is equal or lower than the second highest proportion of information sources.

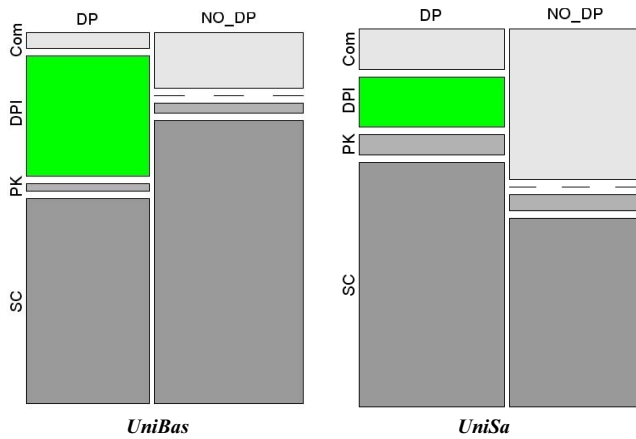


Figure 4 Mosaic plots of source of information

TABLE XIII DESIGN PATTERN SOURCE PROPORTION AND TEST RESULTS

Exp.	Proportion	Relevant	Predominant
UniBas	0.28	Yes (<0.01 , $d=0.49$)	No (0.98, $d=-0.85$)
UniSa	0.14	No (0.91, $d=-0.70$)	No (0.99, $d=-3.12$)

To assess the relevance and predominance of design pattern instances, we tested the proportion of participants using them (when present) by employing a proportion test [1]. Regarding the relevance, we compared the proportion to the average proportion, while, for predominance, we compared the proportion to the second highest proportion. The results are reported in Table XIII. For each experiment, the table shows the proportion and the results of the Mann-Whitney test employed to test whether DPI is relevant or predominant.

The results indicated that for UniBas the proportion is about 30% and the use of design pattern instances as source of information is relevant, but not predominant. The results also suggest that when design patterns are textually documented in the source code (i.e., UniSa), the proportion is less than 15% and their use is neither relevant nor predominant. Also, these contrasting results may be due to the kind of documentation (i.e., TD and GD) used for design pattern instances. It could be possible that participants provided with graphically documented instances perceived more relevant the use of the instances in the source code comprehension.

C. Survey and Post-experiment Survey Questionnaires

Figure 5 graphically summarizes the answers the participants provided to the post-experiment survey questionnaire. The boxplots are divided according to experiment (UniBas and UniSa) and to the Likert scale used for the answers. Further, it is useful to recall that the participants who used NO_DP answered only the questions from S1 to S5.

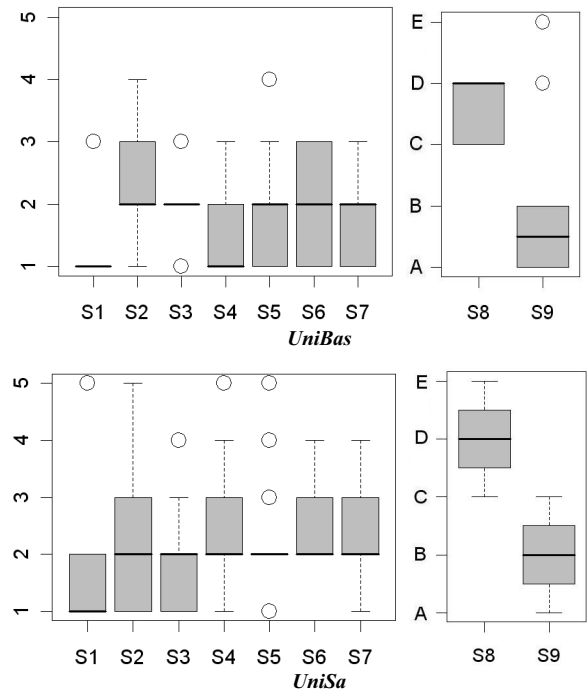


Figure 5 Participants' responses to the post-experiment survey questionnaire

The analysis of the answers for S1 shows that the time needed to carry out the experiments was considered appropriate (see Figure 5) for both the experiments, the median is 1 (*strongly agree*). The medians equal to 2 (*agree*), and the box length and the tails of boxplots of S2 and S3 reveal that the objectives and the tasks were considered to be clear in both the experiments. With regard to S4, the median of the boxplot for the first experiment is 1 (*strongly agree*) while the one for the replication is 2 (*agree*). The distribution of the answers suggests that participants generally found the source code comment clear. Similarly, the participants found useful the organization in three parts of the questions because the median of the boxplot was 2 (*agree*) for both the experiments (S5 in Figure 5).

The median equals to 2 (*agree*), the box length, and the tails of boxplots for the responses to S6 and S7 suggest that the design pattern instances were considered useful to answer the questions and well documented in the system. Regarding S8, we can observe that the median is equal to D and the box ranges from C to D for the first experiment and no tails are present. The median is also equal to D for the replication, but the tails reach C and E. Thus, we can observe that the participants specified that in median the time spent to analyze the source code ranges from 60% to 80% of the total time. Regarding S9, the participants to UniBas and UniSa specified that the time to consult the pattern instances ranged from 20% to 40% of the total time.

D. Discussion of the Results

The results of this study provide evidence that participants achieved better Comprehension and Efficiency values when they had documented design pattern instances as complementary information to the source code. The participants also spent less time to accomplish the comprehension task (i.e., lower Effort values were observed). In case the participants were provided with a UML graphical based documentation their Effort and Efficiency values were significantly better than those of the participants provided with source code alone. The participants achieved better performances also using textually documented design pattern instances. The effect of this kind of documentation is not statistical significant.

We also observed that the way in which the instances were documented did not affect the comprehension of the source code. Differently, the participants that used textually documented instances spent significantly less time to accomplish the comprehension task. Both the results suggest that: it is useful to document the implemented design patterns, but how these are documented may significantly affect the effort to comprehend the source code and the efficiency with which a maintainer comprehends it.

One of the more remarkable results is that the use of the design pattern based development can increase the comprehension of source code only in case the instances are correctly identified. In fact, the results (see the analysis of Hn3_X and Hn4_X) indicated that the capability of correctly identifying the design pattern instances influenced the participants' performances. The results also suggest (see Table X and Table XI) that the capability of the participants

to correctly identify design pattern instances impacted more than the type of representation employed to document them.

A possible justification for these results could be: if a maintainer misunderstand or does not correctly identify design pattern instances, he/she will do a poor job. Thus, it seems crucial to properly document design pattern instances to let maintainers identify them in an easier and more effective way. Future empirical investigations are needed to confirm or contradict the results.

Regarding the source of information, we observed that the participants in both the experiments indicated as first and second source of information the source code and the design pattern instances, respectively. This result indicates that the participants trusted the provided additional information in the execution of comprehension task. Further, the use of design pattern instances resulted to be relevant but not predominant for UniBas, while the use of the instances is neither relevant nor predominant for UniSa. A possible explanation is that participants found easier and/or more attractive the use of graphically documented instances.

The descriptive statistics indicate that the participants to UniSa spent less time to accomplish the comprehension task with DP and NO_DP with respect to the participants to the original experiment. This result could be due to: (i) the participants to UniSa did not have to go back and forth between diagrams (i.e., the documented design pattern instances) and code and (ii) the participants considered the textually documented design pattern instances useless and not very much attention was paid to them. The latter point could also explain why there is no big difference between having textual documentation and no documentation at all.

V. THREATS TO VALIDITY

Conclusion validity concerns issues that affect the ability of drawing a correct conclusion. In our study, we used proper statistical tests. In particular, a non-parametric test (i.e., Mann-Whitney test for unpaired analyses) was used to statistically reject the null hypotheses. When we rejected the null hypotheses, the p-values were mostly less than 0.01. This gives strength to the achieved results.

Internal validity threats are mitigated by the design of the experiment. Each group of participants worked only on one task, with or without the design patterns instances. Another possible threat concerns the exchange of information among the participants. We prevented that monitoring the participants. Fatigue is another possible threat for internal validity. We mitigate the fatigue effect allowing the participants to take a break.

Construct validity may be influenced by the metrics used and by social threats. The employed metrics were widely used in the past with purposes similarly to ours [16]. Regarding the Comprehension variable, one of the authors, not involved in the definition of the task, built the questionnaire to be complex enough so as not to be obvious. To avoid apprehension, we did not evaluate the students on their Comprehension, Effort, and Efficiency values. Finally, a further threat could be related to the modification of the identifiers in the source code used by the participants while accomplishing the comprehension tasks with NO_DP.

External validity concerns the generalization of the results. These threats are related to the comprehension task and to the use of students as participants. Regarding the first point, we selected a part of an open software system large enough to be considered not excessively easy. Another possible threat related to the task is that JHotDraw is well design and implemented. In case of poor designed and documented systems, we could achieve results different from the ones discussed here. On the use of students as participants, we can say that they were specifically trained on software engineering and object-oriented programming tasks. Therefore, we think that they should not be inferior to professional junior developers. It could be also possible that the participants are better trained on design patterns than many professionals of small/medium software companies. Finally, a possible threat to the external validity could be related the students of UniBas and UniSa whose background could be slightly different. However, the data (see Table III and Table IV) indicate that such a difference did not affect the source code comprehension when participants performed the task both using and not using the documentation of the design pattern instances.

VI. CONCLUSION AND FUTURE WORK

The results of this empirical investigation provided evidence that maintainers achieved better comprehension of the source code when design pattern instances are provided as a complement to the source code. In particular, the participants to the original experiment that accomplished the task with graphically documented design pattern instances achieved on average a better comprehension of the source code than the participants that accomplished the tasks without the documentation of the design pattern instances. The effect of the documentation is statistically significant on the time and on the efficiency to accomplish the comprehension task. The effect of the documentation for the design pattern instances was not statistically significant in the replication. However, the descriptive statistics showed that the participants achieved on average better performances in case the instances were available.

To better investigate the impact of the type of the documentation, we also analyzed the data of the two experiments together. The analysis revealed that textually documented design patterns reduced the time needed to accomplish the comprehension tasks as compared with graphically documented design patterns. The results also indicated that the correct identification of the design pattern instances to be used in the comprehension of source code impacted more than the type of representation (i.e., textual or graphical) for documenting these instances. Thus, the main result of our investigation indicated that design pattern based development increases source code comprehensibility only in case the design pattern instances are correctly identified. Accordingly, it is crucial to use representations that make easier the identification of design pattern instances.

Finally, the results also indicated a relationship between the results achieved by the participants and the design pattern instances within the source code. Further empirical investigations are needed to confirm or contradict the results.

REFERENCES

- [1] A. Agresti, *An Introduction to Categorical Data Analysis*, 2007. Wiley-Interscience.
- [2] V. R. Basili, F. Shull and F. Lanubile, "Building Knowledge through Families of Experiments," *IEEE Transactions on Software Engineering*, 25(4), IEEE Press, pp. 456-473, 1999.
- [3] K. H. Bennett and V. T. Rajlich, "Software maintenance and evolution: a roadmap". In *Procs of the Conference on The Future of Software Engineering*, ACM Press, 2000, pp. 73-87.
- [4] J. Bieman, G. Straw, H. Wang, P. W. Munger, R. T. Alexander, "Design patterns and change proneness: An examination of five evolving systems". In *Procs of the International Software Metrics Symposium*, IEEE CS Press, 2003, pp. 40-49.
- [5] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd Edition, Lawrence Earlbaum Associates, 1988.
- [6] W. J. Conover, *Practical Nonparametric Statistics*. Wiley, 3rd Edition, 1998.
- [7] M. Di Penta, L. Cerulo, Y.-G. Guéhéneuc, G. Antoniol, "An empirical study of the relationships between design pattern roles and class change proneness". In *Procs of the International Conference on Software Maintenance*, IEEE CS Press, 2008, pp. 217-226.
- [8] E. Gamma, R. Helm, R. E. Johnson, J. M. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1st Edition, November 1994.
- [9] Y.-G. Guéhéneuc, G. Antoniol, "DeMIMA: A Multilayered Approach for Design Pattern Identification", *IEEE Transactions on Software Engineering*, 34(5), IEEE Press, pp. 667-684, 2008.
- [10] S. Jeanmart, Y.-G. Guéhéneuc, H. Sahraoui, N. Habra, "Impact of the Visitor pattern on program comprehension and maintenance". In *Procs. of the Intl. Symposium on Empirical Software Engineering and Measurement*, IEEE CS Press, 2009, pp. 69-78..
- [11] JHotDraw, <http://www.jhotdraw.org>, (last access, 2011).
- [12] V. Kampenes, T. Dybå, J. Hannay, I. Sjøberg, "A systematic review of effect size in software engineering experiments". *Information and Software Technology*, 49(11-12), Elsevier, pp. 1073-1086, 2007.
- [13] F. Khomh, Y.-G. Guéhéneuc, "Do design patterns impact software quality positively?" In *Procs of the Conference on Software Maintenance and Reengineering*, 2008, pp. 274-278. IEEE CS Press.
- [14] G. Porras, Y.-G. Guéhéneuc, "An empirical study on the efficiency of different design pattern representations in UML class diagrams". *Empirical Software Engineering* 15, 2010, pp. 493-522. Springer.
- [15] L. Prechelt, B. Unger-Lamprecht, M. Philippsen, W. Tichy, "Two Controlled Experiments Assessing the Usefulness of Design Pattern Documentation in Program Maintenance". *IEEE Transactions on Software Engineering*, 28(6), IEEE Press, pp. 595-606, 2002.
- [16] F. Ricca, M. Di Penta, M. Torchiano, P. Tonella, M. Ceccato, "How Developers' Experience and Ability Influence Web Application Comprehension Tasks Supported by UML Stereotypes: A Series of Four Experiments". *IEEE Transactions on Software Engineering*, 36(1), IEEE Press, pp. 96-118, 2010.
- [17] F. Ricca, G. Scanniello, M. Torchiano, G. Reggio, E. Astesiano, "On the effectiveness of screen mockups in requirements engineering: results from an internal replication". In *Procs. of Symp. on Empirical Software Engineering and Measurement*, ACM Press, 2010.
- [18] G. Scanniello, C. Gravino, M. Risi, G. Tortora, "A controlled experiment for assessing the contribution of design pattern documentation on software maintenance". In *Procs. of Symp. on Empirical Software Engineering and Measurement*, ACM Press, 2010.
- [19] E. B. Swanson, "The dimensions of maintenance". In *Procs. of the Intl. Conference on Software Engineering*, IEEE CS Press, 1976, pp. 492-497.
- [20] M. Vokac, "Defect frequency and design patterns: An empirical study of industrial code". *IEEE Transactions on Software Engineering*, 30 (12), IEEE Press, 2004, pp. 904-917.
- [21] C. Wohlin, P. Runeson, M. Host, M. C. Ohlsson, B. Regnell, and A. Wesslen, *Experimentation in Software Engineering - An Introduction*, 2000. Kluwer Academic Publishers.